

計算機科学基礎実験 レポート 2

氏名：加藤悠菜 / 学籍番号：6325040

2026 年 5 月 20 日

1 課題 1:2 進数の個数

1.1 実装

1.1.1 アルゴリズムの説明

与えられた n を 2 進数で表記した際の 1 の個数を求めるため、再帰関数 `count_ones` を実装した。このアルゴリズムは、 $n \pmod{2}$ によって最下位ビットが 1 であるか確認し、その後 $n/2$ を再帰的に呼び出すことで、全ての桁を調べることを可能とした。

1.1.2 プログラム

```
let rec count_ones n =  
  if n < 0 then failwith "Error"  
  else  
    match n with  
    | 0 -> 0  
    | _ -> (n mod 2) + count_ones (n / 2)  
;;
```

1.2 実行過程

私の学籍番号の下二桁である $n = 40$ を例に実行過程を説明する。

1. `count_ones 40` $\rightarrow 40 \pmod{2} = 0$ を保持し、`count_ones 20` を呼び出し。
2. `count_ones 20` $\rightarrow 20 \pmod{2} = 0$ を保持し、`count_ones 10` を呼び出し。
3. `count_ones 10` $\rightarrow 10 \pmod{2} = 0$ を保持し、`count_ones 5` を呼び出し。
4. `count_ones 5` $\rightarrow 5 \pmod{2} = 1$ を保持し、`count_ones 2` を呼び出し。
5. `count_ones 2` $\rightarrow 2 \pmod{2} = 0$ を保持し、`count_ones 1` を呼び出し。
6. `count_ones 1` $\rightarrow 1 \pmod{2} = 1$ を保持し、`count_ones 0` を呼び出し。
7. `count_ones 0` $\rightarrow n = 0$ より返り値 0 を返し、合計は $0 + 0 + 0 + 1 + 0 + 1 + 0 = 2$ となる。

商は「次の桁への移動」、余りは「現在の桁の 1 の判定」という役割を担っている。

2 課題 2: べき乗 n^n の計算

2.1 n^n の値を求める

2.1.1 アルゴリズムの説明

このプログラムでは正の整数 n を引数に取り、 n の n 乗を計算する。 n^n は「 n を n 回かける」という操作であるため、再帰を用いる場合は「現在何回かけたか」を管理するカウンターが必要となる。しかし、引数は 1 つしかないため、関数内部に補助関数 (`aux`) を定義した。

2.1.2 プログラム

```
let rec power_val n=  
  let rec aux count=  
    if count = 0 then 1  
    else n * aux (count - 1)  
  in  
  aux n  
;;
```

2.2 呼び出し回数を数える

2.2.1 アルゴリズムの説明

`power_val` で実装した n の乗算ではなく、「1 を足し合わせる」という操作に置き換わる。再帰の各ステップで $1 + \text{aux}(\text{count} - 1)$ を行うことで、関数の呼び出し回数をカウントしている。

2.2.2 プログラム

```
let rec power_steps n=  
  let rec aux count=  
    if count = 0 then 1  
    else 1 + aux (count - 1)  
  in  
  aux n  
;;
```

3 課題 3: コラッツ予想

コラッツ予想に基づき、任意の正の整数 n が 1 になるまでの操作を実装した。基本規則は以下である。

- n が偶数の場合: $n \leftarrow n/2$
- n が奇数の場合: $n \leftarrow 3n + 1$

3.1 ステップ回数を求める

3.1.1 アルゴリズムの説明

関数 `collatz_steps` は 1 になるまでの操作回数を求める。再帰の各ステップで、 $n = 1$ でなければ操作回数として「1」を足し、規則に基づいた次の値を引数として再帰呼び出しを行う。これにより、 $n = 1$ までの操作回数がカウントされる。

3.1.2 プログラム

```
let rec collatz_steps n=  
  if n=1 then 0  
  else if n mod 2 = 0 then  
    1 + collatz_steps (n/2)  
  else  
    1 + collatz_steps (3 * n + 1)  
;;
```

3.2 推移のリストを求める

3.2.1 アルゴリズムの説明

関数 `collatz_path` は計算過程の数値を順に並べたリストを求める。リスト構成子 (`::`) を使い、現在の値 n をリストの先頭に追加しながら再帰呼び出しを行う。これにより、初項から 1 に至るまでの過程が保存される。

3.2.2 プログラム

```
let rec collatz_path n=  
  if n=1 then [1]  
  else if n mod 2 =0 then  
    n :: collatz_path (n/2)  
  else  
    n :: collatz_path (3 * n + 1)  
;;
```

4 課題 4: べき乗とコラッツ予想に関する考察

4.1 「入力値の大きさ」と「ステップ数」の関係: べき乗 vs コラッツ予想

$n = 7, 8, 9, 10$ の範囲でそれぞれのステップ数を調べた。

入力 n	べき乗 (power_steps)	コラッツ予想 (collatz_steps)
7	8	16
8	9	3
9	10	19
10	11	6

表 1 入力値とステップ数の比較

この結果から、ステップ数の性質には違いが見られる。「べき乗のアルゴリズム」においては、ステップ数は常に $n + 1$ であり、入力値に対して比例的に増加する。これは動作が完全に予測可能であることを示している。

対して、「コラッツ予想」のステップ数は入力に対して極めて不規則である。例えば、 $n = 8$ は $n = 7$ よりも大きい入力であるにも関わらず、ステップ数は 16 から 3 へと大幅に減少している。これは $8 = 2^3$ であり、「偶数なら 2 で割る」というルールのみが適用され続けるためである。このように、コラッツ予想においては入力の大きさよりも、その数値が「2 のべき乗に近いか」や「奇数ルール ($3n + 1$)」によって数値がどれだけ増大するか」がステップ数に影響を与えることが分かった。

4.2 数値の「合流」と収束の仕組み

`collatz_path` を実行した際の結果を比較すると、異なる初期値からスタートした数値が、計算の途中で同じ数値に到達し、それ以降全く同一の経路を通っていた。これが「合流」である。

例えば、 $n = 9$ の推移を見ると、途中で 10 に到達している。

- `collatz_path 9` → [9; 28; 14; 7; 22; 11; 34; 17; 52; 26; 13; 40; 20; **10**; 5; 16; 8; 4; 2; 1]
- `collatz_path 10` → [**10**; 5; 16; 8; 4; 2; 1]

この時点で $n = 9$ のルートは $n = 10$ のルートに合流したと言える。また、 $n = 7$ や $n = 3$ など、最終的には $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ という「2 のべき乗のサイクル」に吸い込まれる形で収束していく。このように 1 度でも経験済みの数値や 2 のべき乗に合流すれば最終的に数値は 1 に収束することがわかった。